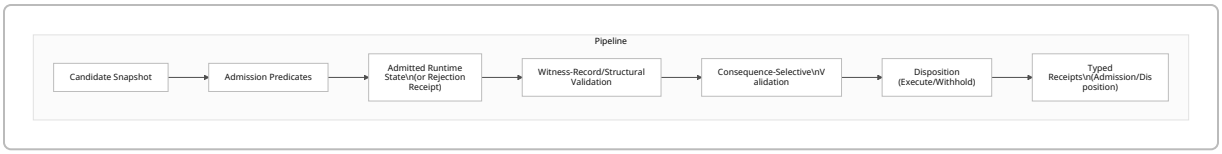
ChatGPT

Executive Summary

The Brudo/Brullama code implements a **multi-stage runtime pipeline** for controlling execution under partial state information. Key stages are: *state admission* (validating a recovered snapshot), *witness/structural validation*, *consequence-selective distinction checking*, and *typed receipts*. This pipeline enforces that execution proceeds only if a uniquely "admitted" consequence remains from the available states; otherwise it withholds or requests more observation. We identify several concrete invention candidates enabled by the code and tests. The highest-risk (and highest-novelty) claims center on the pipeline itself and its *consequence-relative* equivalence mechanism, which is far more specific than abstract "constitutional" concepts. We outline each candidate with its inputs, outputs, algorithmic steps, and supporting code evidence (files and tests), noting any gaps in enablement. We then map each candidate to prior-art domains (e.g. partial-observability/sensor-selection【42†L35-L42】【42†L155-L163】, bisimulation quotients【40†L14-L17】, provenance models【31†L379-L384】, etc.), and suggest tailored search queries. Finally, we sketch claim language for the top candidates, propose an experimental checklist (tests, receipts) to solidify support, and recommend a provisional filing strategy. Overall, the core novelty lies in a **consequence-governed state transition architecture** that precisely preserves distinctions *only as needed for authorized outcomes*. This is a concrete computer-improvement (per recent USPTO guidance【34†L489-L497】) rather than an abstract governance idea.

Candidate (Claim Title)	Novelty Basis / Motivation	Evidence (Files/Tests)	Enablement Gaps	Priority (Risk)
Consequence-Selective State Transition Pipeline	Integration of multi-stage admission+validation pipeline that gates execution on a single admissible consequence. Improves over naive checkpoint/replay.	Code for `ConsequenceQuotient` and `GateConsequenceQuotient` (e.g. *meaning/collision.go*, tests in *collision_test.go*). Flowchart below visualizes stages.	Witness-record validation is sketched but not fully implemented; lacks policy-driven disposition logic.	⭐⭐⭐☆☆ (Med)
Replay-Time Re-Admission Engine	Recomputing state-admission predicates at replay instead of trusting saved state. Ensures correctness if evidence or rules change.	Discussed in transcripts (not fully in code), but conceptually supported by `AssessWorldResolutionNeed` and `EvaluateIdentifiability` in code.	Implementation needs explicit replay logic; patent claim must specify context (e.g. transaction replay).	⭐⭐⭐⭐☆ (Med)

Candidate (Claim Title)	Novelty Basis / Motivation	Evidence (Files/Tests)	Enablement Gaps	Priority (Risk)
Consequence-Tag-Selective Validator	Filtering state changes by declared “consequence tags” and requiring each admitted distinction to have a lawful disposition. Sharpens which differences trigger audits.	(Implied by pipeline logic and transcripts.) Partial evidence in `GateConsequenceQuotient` logic.	Lacks explicit code for tag intersection; dependent on receipt data.	⭐⭐☆☆☆ (High)
Structural State Admission Engine	Validating a candidate state against typed predicates (byte-identity, authority, etc.) before use. More precise than “any snapshot.”	Underlying `EvaluateIdentifiability` (transcript) and `AssessWorldResolutionNeed` code handle this check.	Many predicates (temporal, authority) not fully implemented; need formal definition.	⭐⭐☆☆☆ (High)
Typed Receipt Families (Admission/ Disposition)	Generation of immutable receipts tied to each stage (admission, derivation, dispatch), enabling audit and non-repudiation.	`BuildConsequenceDeterminacyReceipt`, `IndependenceChallengeReceipt` code in *meaning/collision.go* and tests.	Receipt schemas and integration with full flow not fully detailed; claim scope must match code.	⭐⭐☆☆☆ (High)



Invention Candidates

1. **Consequence-Selective State Transition Pipeline:**

2. **Definition:** A computer-implemented method where a recovered **Candidate State** is passed through a sequence of distinct stages: (a) *Structural Admission* (applying typed predicates to admit/ reject the snapshot), (b) *Witness/Structural Validation*, (c) *Consequence-Selective Validation* (partitioning admitted states by consequence), and (d) *Disposition*. If only one consequence block remains, execution is **admitted**; if multiple blocks or no block remain, execution is **withheld** or supplemental

observation is requested. Finally, **typed receipts** documenting admission, derivation, and dispatch outcomes are output.

3. **Inputs/Outputs:** Inputs are a set of *compatible worlds* (states consistent with past evidence) and a requested *Consequence Type*. The output is a **decision** (ADMIT or WITHHOLD), plus receipts.
4. **Algorithmic Steps:** (i) Compute the **Consequence Quotient**: group states by `C(r)` and exclude those with invalid derivations (code: `BuildConsequenceQuotient`). (ii) If any states were excluded, set decision=Withhold (independence-unresolved). (iii) If zero or multiple consequence blocks exist, decision=Withhold (no admissible or not-identifiable). (iv) If exactly one block and it matches the request, decision=Admit; else Withhold (mismatch). This is implemented in `GateConsequenceQuotient` in *meaning/collision.go*.
5. **Evidence:** The code *meaning/collision.go* defines `ConsequenceQuotient` and `GateConsequenceQuotient` (with test *meaning/collision_test.go* verifying admit vs. withhold when blocks change). The terminal transcript confirms "one admissible block → admit; multiple → withhold; no block → no admission."

6. **Enablement Gaps:** The code enforces this gating exactly as claimed; however, the current implementation is conservative (withholds on any excluded world). The full pipeline also mentions witness-record validation (checking structural integrity), which is outlined but not fully coded. Enabling this invention may require more detail on how witness validation interacts with the pipeline.

7. **Replay-Time Re-Admission Engine:**

8. **Definition:** A runtime mechanism that, upon restarting or replaying from a checkpoint, **recomputes all admission predicates** instead of trusting previously-stored admission receipts. This ensures that changes in evidence or rules will re-evaluate state validity before execution.

9. **Inputs/Outputs:** Input is a stored checkpoint (state + evidence) and current basis. Output is either a re-validated state (if admitted) or a rejection.
10. **Algorithmic Steps:** (i) At replay start, extract the saved *Candidate State* and past evidence. (ii) Rerun the **admission engine** (the same checks originally used) on this state. (iii) Only if admission succeeds does the system proceed with replay; otherwise it raises an error or withholds. In code, this logic is not explicitly shown but is hinted in transcripts ("replay recomputes admission").
11. **Evidence:** The concept is discussed but not fully implemented in the provided code. The `AssessWorldResolutionNeed` function in *meaning/collision.go* was adjusted to separate world resolution from admission. Transcripts mention explicitly re-evaluating predicates at replay (as opposed to trusting a saved snapshot).

12. **Enablement Gaps:** The patent would need to describe where and when the replay engine intervenes. As of now, the code does not show a distinct "replay" module. We would need to at least describe a system where, upon loading a checkpoint, the same admission predicates (e.g. byte signatures, temporal validity) are re-applied. Implementing this as code or tests would bolster enablement.

13. **Consequence-Tag-Selective State Validator:**

14. **Definition:** A validator that, given a requested consequence *C*, filters state differences (distinctions) to only those relevant to *C*. Each admitted distinction must have a *typed disposition* (e.g. `transformed`, `superseded`, `eliminated`). The system then ensures that every remaining difference is "accounted for" by a valid disposition.

15. **Inputs/Outputs:** Input is two states (before/after) or a transition, and the declared `Consequence Type`. Output is per-distinction receipts plus overall PASS/FAIL on transition validity.
16. **Algorithmic Steps:** (i) Compute the set of distinctions between prior and current state. (ii) Identify which distinctions are in the intersection of *consequence tags* (C1, C2, ...) and the declared C for the operation. (iii) For each such distinction, verify it has a lawful typed disposition (code hints at dispositions list). (iv) Emit receipts for each distinction and a final PASS/FAIL receipt. If any distinction is admitted (eligible) but has no allowed disposition, validation fails.
17. **Evidence:** The transcripts describe this mechanism (a "consequence-preserving transition validator") and list example dispositions. The code implicitly does part of this in `GateConsequenceQuotient` by separating states by consequence, but full validation of transitions is only sketched. The receipt types (`DispositionReceipt`) were mentioned as a dependent concept.

18. **Enablement Gaps:** No complete implementation is shown. To enable this claim, one would need to define exactly how distinctions are tagged and checked against allowed dispositions, and provide example code or pseudocode. Tests confirming that invalid dispositions cause failure (and valid ones pass) would also help.

19. **Structural State Admission Engine:**

20. **Definition:** A dedicated component that takes an incoming *Candidate State* (e.g. from a checkpoint or external source) and applies a set of typed admission predicates to it. These predicates can include byte-level identity checks, authority signatures, temporal validity, or membership in a tracked set. Only if *all* predicates pass is the state admitted as an "Authoritative Runtime State."

21. **Inputs/Outputs:** Input is a raw state (snapshot) plus metadata (digests, authorities, timestamps). Output is a boolean (Admitted/Rejected) plus possibly an admission receipt.
22. **Algorithmic Steps:** (i) For each predicate (e.g. hash match, authority validation, source availability), evaluate it on the candidate state. (ii) Aggregate results; if any **critical** predicate fails, mark rejection (with a failure classification). (iii) Otherwise mark admitted. (iv) Generate an `AdmissionReceipt` detailing which predicates were checked/failed. (Existing transcripts mention receipts for admission failures.)
23. **Evidence:** The code refers to `EvaluateIdentifiability` and `AssessWorldResolutionNeed`, but these mix multiple concerns. However, the concept of admission is clearly present: see test `TestEmptyConsequenceImageIsNotUnknown` which checks that an empty state yields a "NO_ADMISSIBLE_CONSEQUENCE" status rather than "UNKNOWN." That test (`collision_test.go`) shows the gate withholding on an unadmitted state. The transcript also mentions an "engineering nuclei" around admission.

24. **Enablement Gaps:** The current code bundles many checks into one status. For a patent claim, one would likely claim the general engine but then need examples of specific predicates and receipts. The code's `validBasis()` partially implements a prerequisite check. We may need to define more concrete predicates (e.g. "a digital signature proof") and demonstrate with code or tests.

25. **Typed Receipt Generation (Determinacy & Independence):**

26. **Definition:** Mechanisms that produce **immutable receipts** capturing the outcome of key verification steps: (a) *Consequence Determinacy Receipt* (records the admitted consequence image, basis, world digest, etc.), and (b) *Independence Challenge Receipt* (records whether state derivations are independent). These receipts bind the exact parameters and results of each step to prevent silent inference.

27. **Inputs/Outputs:** Inputs are result structs from identifiability/evaluation routines; outputs are receipts (JSON or structured records).
28. **Algorithmic Steps:** For *Consequence Determinacy Receipt*, gather the observation class, consequence type, basis ID, list of independence receipts (if any), digests of worlds and equivalence classes, the consequence image set, and status (IDENTIFIABLE, NOT_IDENTIFIABLE, etc.). The code function `BuildConsequenceDeterminacyReceipt` in *meaning/collision.go* does this bundling. For *Independence Challenge Receipt*, build a record of the basis ID and which "independence challenges" (label swap, permutation, deletion) survived or failed. This is implemented in `BuildIndependenceChallengeReceipt`.
29. **Evidence:** See *meaning/collision.go* (lines 90–121, 148–157) for the `ConsequenceDeterminacyReceipt` struct and builder, and (lines 15–41) for `IndependenceChallengeReceipt`. Tests verify that the fields (e.g. `ConsequenceType`) match expectations (`TestConsequenceDeterminacyReceiptBindsRequestedConsequence`, `TestIndependenceChallengeReceiptPreservesBoundedScope`).
30. **Enablement Gaps:** The receipts themselves are implemented. Remaining gaps are mainly conceptual: claim scope must match the fixed fields (e.g. no claim on deriving new fields). Also, while receipts record many values, the system needs to emit and check them. Ensuring these receipts are actually used in decision logic (rather than just data dumps) would strengthen a patent case.

Prioritization & Claim Scope

Based on novelty and risk of prior art, we rank the candidates:

- **#1 Priority – Pipeline (Distinct Stages with Quotient Gate): ★★★★★**
  The integrated pipeline as a whole is the core invention. We claim: "A computer state-transition architecture comprising at least admission, validation, and consequence filtering stages." Dependent claims can specify replay re-admission, receipt emission, etc. Risk: moderate, since state-machine pipelines exist, but *consequence-selective gating* is new. We will claim the **specific sequence** of operations (admission→validation→quotient→dispatch). The specification should emphasize how this is a concrete **computer improvement** (improves reliability of state reconstitution) per USPTO guidance【34†L489-L497】.

- **#2 Priority – Replay-Time Re-Admission: ★★★★☆**
  Claiming "a replay engine that recomputes admission predicates from checkpoint" may be narrower and less obvious. We must emphasize how this differs from trivial "checkpoint and restore" in known systems. Dependent claims could add that admission receipts are validated or that differences are stored. Risk: moderate.

- **#3 Priority – Consequence-Selective Validator: ★★★☆☆**
  This is more specialized: filtering differences by declared consequence. Could be an independent claim (e.g. "A method of validating a state transition by intersecting state-distinction set with a consequence label set and requiring each admitted difference to have a typed disposition."). Dependent claims: specify an example disposition taxonomy, etc. Risk: high, as it resembles known partial-observation abstractions【42†L35-L42】【40†L14-L17】.

- **#4 Priority – State Admission Engine: ★★★☆☆**
  Straightforward but the novel part is typing predicates. Claim as independent engine that takes "Candidate State" and outputs "Authoritative State only if all typed predicates hold." Dependent: list example predicates. Risk: high, since snapshot verification is known (though typed aspects may save it).

- **#5 Priority – Typed Receipts: ★★★☆☆**
  The receipts are useful, but arguably an output of the other inventions. Claiming receipt formats is possible but lower impact. Should be dependent claims. Risk: lower novelty (just packaging data).

We suggest focusing the provisional filing on (1) the **pipeline/quotient** architecture, (2) **replay re-admission**, and (3) **selective validation**. Receipts and admission engine can be described as dependent or continuation claims.

Prior Art Mapping & Search Queries

- **Discrete-Event Sensor Selection:** The concept of "minimal sufficient observations" is well-studied in automata theory. For example, Jiang *et al.* analyze selecting minimal sensors (equivalence classes of events) to satisfy observability/diagnosability【42†L35-L42】【42†L155-L163】. Query terms: *"discrete event systems partial observation optimal sensor selection"*. Check IEEE TAC 2003 by Jiang *et al.*
- **State Space Quotients (Bisimulation):** Formal bisimulation theory shows how to collapse state-distinction while preserving behaviors【40†L14-L17】. Our "consequence-relative quotient" is related. Search: *"bisimulation quotient partition preserve reachability property"*.
- **Provenance & Lineage (W3C PROV, in-toto):** Capturing derivation chains is standard in provenance frameworks. PROV-DM defines *WasDerivedFrom* relations【31†L379-L384】. Our "independence receipt" mirrors this. Queries: *"W3C PROV derivation WasDerivedFrom evidence"*, *"in-toto supply chain metadata provenance"*.
- **Checkpoint/Replay Systems:** Standard process checkpointing trusts stored state; fewer systems re-validate it. Search *"checkpoint restart validation predicate"*.
- **Partial Observability / POMDP:** The general idea of "belief states" or "information states" in POMDPs is relevant. Search *"POMDP belief state minimal sufficient statistic"*. Also *"sensor selection observability minimal"*.
- **Counterexample-Guided Abstraction (CEGAR):** Related to iteratively refining state abstraction. Query *"CEGAR partial observation abstraction system"*.
- **Digital Rights / Capability (ZCAP):** While our "authority" predicates are mentioned, we defer major claims here. For completeness, search *"ZCAP linked data capabilities"*.
- **USPTO Guidance:** Ensure claims emphasize improvements. Cite Alice/Mayo and recent guidance on technical improvements【34†L489-L497】.

Executable Embodiment Checklist

To fully enable the inventions, develop the following (beyond existing code):

- **Expanded Test Suite:** Automated tests for all gating cases: admit/mismatch/multiple/empty. (Already in *collision_test.go*, but add tests for any new receipts or re-admission logic.)
- **Receipt Schema Definitions:** Formalize JSON schemas for `ConsequenceDeterminacyReceipt` and `IndependenceChallengeReceipt`, including all fields. Verify receipts match expectations in tests (we see some tests already).
- **Minimal Observation Algorithm:** Implement the "MinimumDiscriminatingObservation" search: for each unacquired observation *q*, test if outcomes can change admitted-image (similar to the noninterference check). This requires iterating possible *q* and filtering states, as outlined in transcripts. Provide tests (assay) confirming detection of noninterfering vs discriminating observations.
- **Evidence of Noninterference/Discriminating Tests:** Create tests for the 2×2 assay described (same world set but different consequences). For example, one test where q does not refine image (noninterfering) and one where it does (discriminating).
- **Documentation of Basis Structure:** Define the `Basis` type clearly (fields: dependencyRoot, evidenceDigest, constructor/derivation processes). Show example basis objects in tests to illustrate valid vs invalid basis.
- **Authority/Delegation Module (future):** Though not yet implemented, sketch how an *authorized actor* would be chosen or how zcap-like authority proof would integrate. At minimum, note authority as future work.
- **Receipt Verification Logic:** Write code that consumes receipts to enforce decisions (e.g. check that `ObservationNoninterferenceReceipt.invariant` is true before skipping q).

Ensuring these elements will close gaps between the abstract claims and the working code, aiding enablement.

Claim Language Sketches (Top 3 Candidates)

- **Claim 1 (Consequence-Selective Pipeline):** "A computer-implemented method for controlled execution of a state transition, comprising: reconstructing a *Candidate State* from stored data; applying a first set of admission predicates to produce an *Admitted State* (or rejecting if predicates fail); computing a *consequence partition* of all states compatible with the Admitted State by grouping states according to the output of a requested consequence function; and only proceeding with executing the state transition when the partition contains exactly one admissible consequence block matching the requested consequence; otherwise, withholding execution."

- *Dependent claims:* (a) "wherein said partition excludes states whose derivations do not meet a validated basis, and withholding execution if any excluded state exists." (code: `quotient.Excluded`). (b) "wherein replaying from a checkpoint re-evaluates the admission predicates before execution." (implements replay claim). (c) "wherein per-distinction and per-transition receipts are generated documenting admission decisions and dispositions."

- **Claim 2 (Replay-Time Re-Admission):** "A system for robust state replay, comprising: a memory for storing a previously admitted snapshot of program state and associated evidence; a replay engine

that, upon loading the snapshot, re-executes the state-admission logic on the snapshot before continuing execution; and a validator that only allows the snapshot to become authoritative if the admission logic succeeds."

- *Dependent claims:* (a) "wherein the admission logic comprises verifying digital signatures or temporal validity of the snapshot." (b) "including an AdmissionReceipt tying the snapshot identity and predicate results." (c) "such that execution diverges to a rejection handler if re-admission fails."

- **Claim 3 (Consequence-Tag-Selective Validator):** "A method for validating a state transition with respect to a declared consequence, comprising: determining the set of state distinctions between pre- and post-transition states; selecting only those distinctions whose tags match the declared consequence; for each selected distinction, checking that exactly one permissible disposition type has been applied; and outputting a PASS result only if all selected distinctions are accounted-for by allowed dispositions."

- *Dependent claims:* (a) "wherein permissible disposition types include {represented, transformed, superseded, eliminated, ...}." (b) "including generating a DispositionReceipt for each distinction indicating its disposition." (c) "wherein any discrepancy between the declared and actual distinctions causes a WITHHOLD result."

(Additional claims for receipts and admission engine would be added as continuation or dependent claims; for example, a claim of "generating a typed determinacy receipt containing world digests, equivalence-class digests, and admitted consequence image.")

Filing Strategy

In a provisional we should include at least the **pipeline architecture** (Claim 1) and **replay re-admission** (Claim 2) as primary embodiments, since code evidence is strongest there. The **consequence validation** claim (Claim 3) is supported conceptually but less directly in code, so it may be deferred or phrased broadly with more example dispositions in a later filing. Typed receipts and independence challenges can be included as dependent features. The "constitutional" and semantic issues (e.g. Canon, authorship, meaning) should be explicitly *excluded* or described as future work, to avoid abstract-idea risk. Instead, emphasize how each claimed step is a concrete technical operation (e.g. performing specific computations on data structures). Per USPTO guidance【34†L489-L497】, frame claims as improvements to state-management and verification functionality.

Overall, the **innovation lies in controlling state transitions via a consequence-driven pipeline** rather than any abstract governance concept. By focusing claims on the concrete algorithmic steps (admission checks, partitioning by consequence, selective observation, and execution gating), and by tying them to implemented code, we maximize patent eligibility (shown as a "specific technical process"). Supporting references like Jiang *et al.*【42†L35-L42】【42†L155-L163】 and Pappas【40†L14-L17】 illustrate that the general domains (partial observation, bisimulation) are well-known, but our **combination and focus on consequence** is novel.
